

LEAD: Logarithmic Exponent Approximate Divider For Image Quantization Application

Omkar G Ratnaparkhi

IIIT Bangalore

Bangalore, Karnataka, India

omkar.ratnaparkhi@iiitb.ac.in

Madhav Rao

IIIT Bangalore

Bangalore, Karnataka, India

mr@iiitb.ac.in

ABSTRACT

Most of the applications of modern day VLSI designs are approaching towards energy efficient and high speed computing solutions. Approximate computing is considered a suitable design methodology that satisfies the current requirements of hardware and performance metrics without compromising on the outcome significantly. Many of the arithmetic operations are realized using approximate computing techniques, and many successful implementations are reported at system level designs. However divider operations in general are rarely realized in hardware and this needs much attention considering the surge in neural networks implementation in hardware. In this paper, a novel approximate divider is proposed which is not only characterized to have better accuracy and hardware efficient when compared to the other accurate dividers. The proposed divider is built on logarithmic divider and approximates the exponent part to achieve the desired hardware characteristics. The proposed 8-bit, and 16-bit divider design were realized in 45-NM CMOS technology for different input and output data format including integer, fixed-point, and floating-point. The proposed divider was characterized for error and hardware metrics and compared with other dividers. The novel divider was validated on K-means color quantization algorithm, showcasing improved quantization results.

CCS CONCEPTS

• **Hardware** → **Arithmetic and datapath circuits**; *Combinational circuits*; *Application specific integrated circuits*.

KEYWORDS

Approximate divider, color quantization, exponent series, hardware accelerator.

ACM Reference Format:

Omkar G Ratnaparkhi and Madhav Rao. 2022. LEAD: Logarithmic Exponent Approximate Divider For Image Quantization Application. In *Proceedings of the Great Lakes Symposium on VLSI 2022 (GLSVLSI '22)*, June 6–8, 2022, Irvine, CA, USA. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/3526241.3530323>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

GLSVLSI '22, June 6–8, 2022, Irvine, CA, USA.

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-9322-5/22/06...\$15.00

<https://doi.org/10.1145/3526241.3530323>

1 INTRODUCTION

Most of the mobile and datacentric applications expects energy efficient and high speed computational system design. Semiconductor technology scaling was the major driving factor for designing computationally efficient system, however with limitations posed by the ever shrinking technology node, new approach is desired for designing performance and energy aware system to satisfy the ever growing consumer demands [1–4]. Approximate computing allows to leverage the current CMOS technology, by minimizing the gates but generate inexact output, that is resilient to the changes in the final outcome [5]. This strategy has extended the current CMOS technology life to many more years, and at the same time conserves the hardware metrics including power, delay, and footprint with a compromise in the accuracy [5]. Besides, given the rising demand for various multimedia applications on edge computing device, hardware implementation of image and video processing modules with approximate design technique is considered highly suitable, especially when it is targeted towards low power, and faster computing. Many real time imaging, and video processing applications have relaxed output requirement with no real emphasis on the precision, but conserving the exact outcome [1, 5, 6]. However high loss in the precision [7] hampers the outcome, especially when the hardware accelerator is intended towards autonomous system design.

Many arithmetic operations are realized in hardware using approximate design methodologies in the recent past [8]. Approximate adders and multipliers are studied immensely, and applied to discrete cosine transforms, edge detection, and various other applications, with acceptable outcomes [9–11]. However not many work has been explored on the implementation of division operation [12]. Hardware division is generally ignored since the operation is compute intensive, and essentially loads the overall design. Look-up-Table (LuT) based implementations are generally utilized, however due to the recent emergence of neural network computing and accelerators within the design, the need for division operation in most of the activation functions have come back to the fore. In terms of comparison with multiplier, division generally costs more energy and delay along with its complex architecture. Around 9 cycles are required to complete the division operation when compared to three clock cycles for multiplication operation [13]. Traditionally a single precision divider concedes 35% more hardware resources than multiplier design, and is reported with 27% slower than that of multiplier design [14]. Division operation is broadly implemented in either of five techniques that includes digit recurrence, functional iterations, high radix, lookup table, and variable latency [12]. Digit recurrence is a class of design, that uses shift and subtraction

operations. The popular SRT division falls in this category of design [15]. Functional iteration class involves multiplication instead of subtraction [12]. High radix method is similar to digit recurrence, however the number of quotient digit initialized in one iteration is comparatively high. Variable latency class is similar to previous category, with different quotient bits reinitiated at different iterations. Look-up-table (LUT) type utilizes high memory resources to store the pre-computed values and retrieve the same for estimating the quotient values.

Exact dividers offering precision and range is always a challenge in terms of hardware metrics and resources, and hence approximate dividers with a loss in accuracy is a reliable alternative for realizing hardware efficient accelerator designs. Few approximate dividers were proposed in the past. Truncapp is based on truncated value of dividend, which is then multiplied by the approximate inverse value of the divisor [16]. SEERAD uses rounding of divisor, besides using shifting and add operations to estimate the final division value [17]. LUT based approximate divider proposed in CADE employs subtraction of mantissa part of the input operands and also utilizes first $N - \text{bits}$ of input operands, referred to as tuning bit, to predict division error [18]. Besides the division error was predicted by tuning first N bits of input operands. Hybrid design which combines restoring array with logarithmic divider was proposed in [7]. Most of the dividers mentioned in the literature are either hardware efficient, but concedes substantial error, or vice-versa. Additionally most of the design caters to integer division, hence an hardware efficient design with minimal difference from the true output, and applicable to all different data-formats including integer, fixed-point, and floating-point with similar design blocks, is desirable.

The paper proposes a design that adopts logarithmic divider [7], and minimizes the approximations by employing exponent expansion series. The proposed divider was compared with logarithmic divider and Scalable Accuracy Approximate Divider with Error Compensation (SAADI-EC) [13]. SAADI-EC employed Taylor series expansion to establish multiplication of the inverse of divisor with dividend. A Newton Raphson method to establish inverse of divisor and then multiplying with dividend was also proposed in [10]. The proposed divider is utilized for color quantization scheme which is widely used in image and video processing applications. Color quantization is a regular method employed to compress the image preserving overall appearance and information. K-mean color quantization technique is one of the quantization algorithm which uses mean update step in each of the iterations, thereby an efficient hardware-efficient divider is expected to improve the performance of the overall process. The improvement in the logarithmic divider and characterizing along the design level and at application level is performed for the first time, as per the authors knowledge. The novel divider design is also benchmarked against other dividers to showcase the benefits and drawbacks if any.

2 PROPOSED DESIGN

2.1 Logarithmic divider

Logarithmic divider proposed by [7] inherently employs two approximations in its operations. To elaborate the discussion, consider A and B as dividend and divisor respectively, and both are non

zero positive integers. The dividend and divisor are redefined as $A = 2^{ea} \times (1+a)$, and $B = 2^{eb} \times (1+b)$, where ea and eb are positions of leading one bit in the binary forms, and a , and b are the mantissa that lies in the range of $[0,1]$. The division operation generating quotient (Q) is further expressed in the following set of mathematical representations. $Q = \frac{A}{B} = \frac{2^{ea} \times (1+a)}{2^{eb} \times (1+b)}$. The logarithmic equivalence is considered as $\log_2 Q = ea - eb + \log_2(1+a) - \log_2(1+b)$. The first approximation is applied to establish and simplify the logarithmic expression resulting in $\log_2(1+a) = a$, and $\log_2(1+b) = b$. The logarithmic quotient is represented as $\text{Log}_2 Q = ea - eb + a - b$, and the quotient as $Q = 2^{(ea-eb)} 2^{a-b}$. A second approximation of $2^{(a-b)} = 1 + (a-b)$ further simplifies the divider realization stated in the equation 1.

$$Q = \begin{cases} 2^{(ea-eb)} \times (1+a-b) & \text{for } a-b > 0 \\ 2^{(ea-eb-1)} \times (2+a-b) & \text{otherwise} \end{cases} \quad (1)$$

Overall the division operation realized in [7] requires four subsystem designs involving leading one detector (LOD), logarithmic to binary converter (LBC), subtractor, and binary to logarithmic converter (BLC). The block level representation of division operation is shown in the Figure 1 where 8-bits of mantissa and 3 bits of exponent is realized at the output.

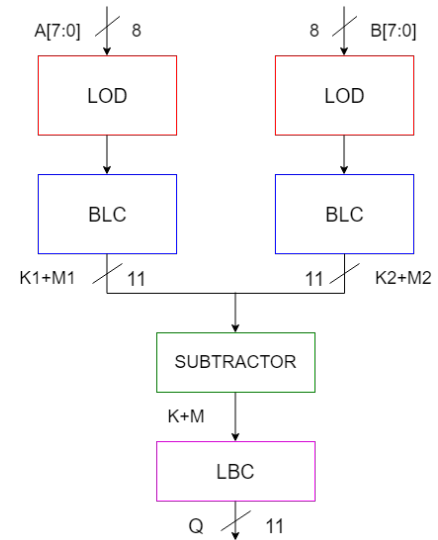


Figure 1: Block diagram representing non iterative logarithmic divider [7].

2.2 Novel divider design

The Logarithmic exponent approximate divider (LEAD) design is proposed for three types of data format: integer, fixed point, and floating point. The former divider is realized only for integer data-format. A divider design incorporating all data-formats is considered highly useful for running different algorithms related to image processing, and hence can be reused efficiently. The proposed LEAD divider design comprises of similar blocks of LOD and BLC to normalize the operands: divisor and dividend, similar to logarithmic

divider. The position of leading one bit is stored in the form of exponent ea and eb respectively. It is expressed as $A = 2^{ea} \times (1 + a)$, and $B = 2^{eb} \times (1 + b)$ where, a and b are in the range of $0 \leq a < 1$ and $0 \leq b < 1$. The subtraction of ea and eb results in the difference in the exponent terms which is eventually employed for shifting the data-bits of the generated normalized quotient. The proposed design redesigns the subtraction result of mantissa a , and b for minimizing the second approximation, and improving the accuracy over the logarithmic divider. The proposed design employs exponent series to yield the result of 2^{a-b} . The exponent series in general form is expressed in the equation 4. Only initial two terms of this series are employed, resulting in sufficiently faster convergence when compared with the current accurate division technique of SAADI-EC considering iterative terms of 7 for 8-bits ($t=7$), and 15 for 16-bits ($t=15$) data. in the literature [14].

$$e^x = 1 \pm \left(\frac{x}{1!}\right) + \left(\frac{x^2}{2!}\right) \pm \left(\frac{x^3}{3!}\right) + \dots \quad (2)$$

$$a^x = e^{x \ln a}$$

$$2^x = 1 \pm (x \ln 2) + (x \ln 2)^2 \times 0.5 \pm (x \ln 2)^3 \times 0.1667 + \dots \quad (3)$$

$$2^x = 1 \pm (x \times 0.693) + (x \times 0.693)^2 \times 0.5 \pm (x \times 0.693)^3 \times 0.1667 + \dots \quad (4)$$

Hence the overall quotient generated from the proposed LEAD divider is stated in the equation 5.

$$Q = 2^{(ea-eb)} \times (1 \pm (0.693(a-b)) + 0.5 \times (0.693(a-b))^2) \quad (5)$$

The result generated from the exponent series is utilized to establish the mantissa, and used along with the resultant exponent, to generate the final quotient in the fixed point representation. The datapath design of the proposed LEAD divider shown in the

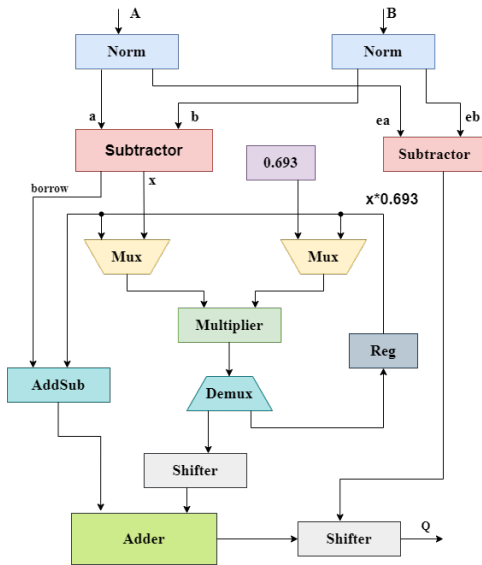


Figure 2: Block diagram representing datapath design for the proposed LEAD divider.

Figure 2, presents sequential flow to complete the division operation. The blocks referred to as *Norm*, *Reg*, *Demux*, *Mux*, and *Add/Sub* refers to Normalizer, Register, Demultiplexer, Multiplexer,

and adder-or-subtractor blocks respectively. The multiplexer unit controls re-using of multiplier block to generate $(0.693 \times (a - b))^2$ term. A timed data-flow generating the quotient result is shown in the Figure 3, where the division operation is completed in two clock cycles. The arithmetic units of normalizer, subtractor, adder, multiplier, registers, and shifter are used along with Look-up-table (LUTs) to generate the division result. Figure 3 shows the vertically stacked hardware blocks, and the flow of the data operations are captured horizontally in 2 clock cycles. The design shows two input operands dividend and divisor represented as A and B respectively, ea and eb as exponent terms which is calculated by estimating the position of leading "1" of dividend and divisor. Both A and B are normalized to a and b respectively by left-shifting to the most significant bits (MSB), and truncating to n bit-width. Two normalizer blocks are positioned in the first cycle parallelly and the output of the normalizer block is fed to the subtractor module. The output of the subtractor block is fed to multiplier block which seeks another operand from LUT to generate $x \times 0.693$ result. Depending on the borrow generated by subtractor block, addition or subtraction is performed by add-sub block, to yield $1 \pm (x \times 0.693)$, where x is considered $a - b$. The multiplier output generated in the first clock cycle ($x \times 0.693$) was utilized in the subsequent clock cycle to generate $(x \times 0.693)^2$. The square result obtained from multiplier block is of $2n$ bit-width, for an n bit multiplier operands. The $2n$ bit-width is further truncated to n bit-width by considering the first n bits of the results. The square result is subsequently left shifted by a bit to generate $0.5 \times (x \times 0.693)^2$ result. The Adder subsystem yields the final mantissa output, which is fed to the shifter subsystem to get the final quotient. For floating-point data, quotient in floating-point is generated by utilizing mantissa and exponent term. A detailed example to explain the sequenced flow of the divider operation is discussed below in the Table 1. Here we consider an example of $190 \div 11$ for 8-bit operands. An approximate Quotient of 17.3028 is achieved by the 8-bit design, and is close to the accurate quotient of 17.2727 .

3 RESULTS AND DISCUSSIONS

The proposed LEAD design for different input data-format as listed below were designed and evaluated.

- 8-bit input integer data format,
- 16-bit input integer data format,
- 16-bit fixed-point data-formatted input representing 8-bit of integer and 8-bit of fractional part,
- 16-bit floating point data with 12-bit mantissa and 4-bit exponent.

The 8-bit divider design output is formatted to represent 8-bits of mantissa and 3-bits of exponent as discussed in SAADI-EC design [14]. The 16-bit divider design for integer and floating-point format is realized with output format similar to input format. All these divider design were realized in 45 nm technology node using Cadence Genus tool. Standard configuration of 1V rail voltage, and room temperature (25°C) were considered for the divider designs. Silicon area, delay, power and energy consumed were realized for all the designs. The hardware metrics of the proposed LEAD design were compared with the earlier logarithmic divider [19], and SAADI-EC of integer data-format [14]. SAADI-EC of 7 iterative

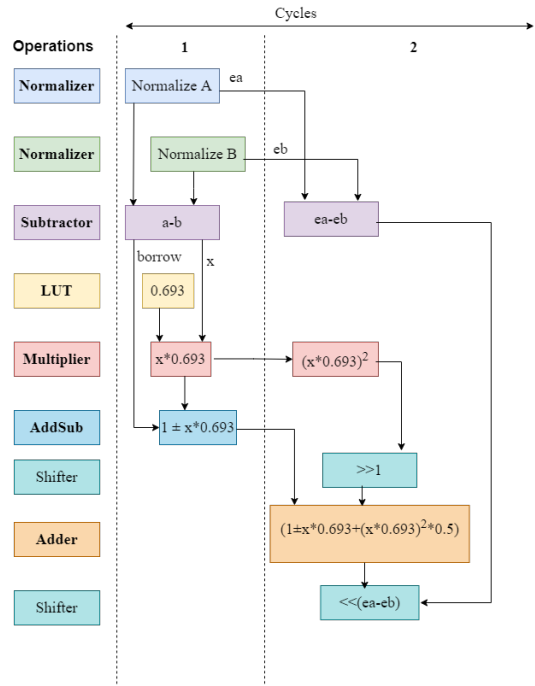


Figure 3: Block diagram representing hardware blocks used for timed data flow.

Table 1: Example showing the *LEAD* division operations for $190 \div 11$.

Operations	Terms	Values
Normalization	$A = (2^{ea}(1+a))$	$2^7 \times (1 + 0.484375)$, $ea = 7$, $a = 0.484375$
Normalization	$B = (2^{eb}(1+b))$	$2^3 \times (1 + 0.375)$, $eb = 3$, $b = 0.375$
Subtractor	$\log_2 Q = (ea - eb) + (a - b)$ $Q = (2^{ea-eb} \times 2^{a-b})$	$7 - 3 + \log_2(1 + 0.484375) - \log_2(1 + 0.375)$ $2^4 \times 2^{0.484375 - 0.375}$
Multiplier	$(a - b) \times 0.693$	$0.109375 \times 0.693 = 0.0757$
Multiplier	$((a - b) \times 0.693)^2$	$0.0757 \times 0.0757 = 0.00573$
Shifter	$((a - b) \times 0.693)^2 \times 0.5$	0.002865
AddSub	$1 + ((a - b) \times 0.693)$	1.0757
Adder	$1 + ((a - b) \times 0.693) + ((a - b) \times 0.693)^2 \times 0.5$	1.08143
Shifter	Q	$2^4 \times 1.08143 = 17.3028$

terms ($t=7$), and 3 iterative terms ($t=3$) for 8-bits design, besides 15 iterative terms ($t=15$) for 16-bits design were implemented. The error probability distribution of the novel *LEAD* divider, along with SAADI-EC with 7 iterations ($t=7$), and logarithmic divider were evaluated for all combinations of 8-bit operands and is presented in the Figure 4. Error distribution plot clearly depicts that the output

errors generated by the Logarithmic shows wide range on the positive side, whereas the SAADI ($t=7$) with error-correction shows higher frequency of smaller errors. The range of error is minimal for SAADI-EC. The *LEAD* divider showcases a symmetrical distribution of positive and negative errors, which is more feasible considering the lower magnitude of errors. The error metrics in the form of mean absolute error (MAE), and root mean square error (RMSE) were also evaluated for these divider designs. The hardware metrics and error metrics of 8-bit, and 16-bit divider designs are mentioned in the Table 2, and 3 respectively.

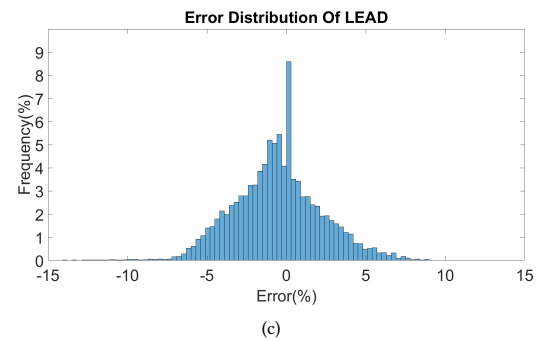
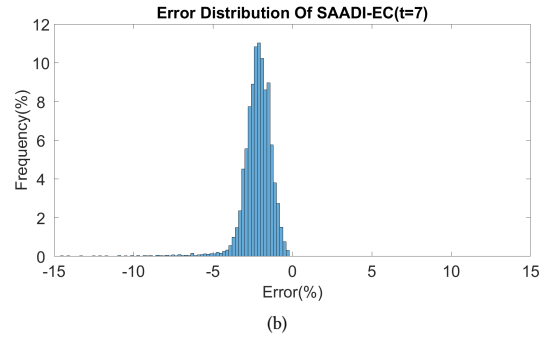
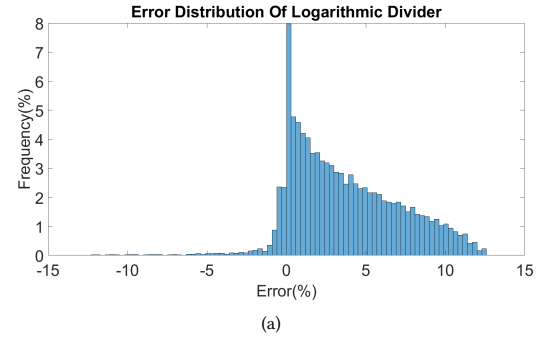


Figure 4: Error distribution characterized for 8-bits (a) Logarithmic divider, (b) SAADI-EC divider, and (c) Proposed *LEAD* divider.

The *LEAD* design with single multiplier block utilizes additional finite state machine design for achieving datapath flow, and extra logic blocks such as multiplexers, and registers to store data and is bound to have larger footprint than Logarithmic design. The *LEAD*

Table 2: Characterized hardware design metrics and error metrics for 8-bit divider designs.

Divider	Area (μm^2)	Power (mw)	Delay (ns)	Energy (nJ)	MAE	RMSE
Logarithmic	198.702	0.003278	2.183	0.00715	0.0392	0.0523
LEAD	1775.271	0.42815	6.82	2.92	0.0225	0.0326
SAADI-EC(t=7)	3035	0.67	8.54	5.72	0.0223	0.0303

Table 3: Characterized hardware design metrics and error metrics for 16-bit divider designs.

Divider	Area (μm^2)	Power (mw)	Delay (ns)	Energy (nJ)	MAE	RMSE
Logarithmic	599.526	0.0099	3.835	0.0379	0.0448	0.0554
LEAD	5482.038	0.7283	9.907	7.2152	0.0218	0.0280
SAADI-EC(t=15)	6105	2.11	23.55	49.71	0.0028	0.0081
LEAD-fixed	5829.065	0.8159	10.052	8.2014	0.0249	0.0485
LEAD-floating	2138.047	0.2855	9.515	2.7165	0.0210	0.0270

for 8-bit design offers 42.6% improvement in MAE, and 37.66% benefits in RMSE, compared to the Logarithmic divider as shown in the Table 2. The error metrics are comparable to the SAADI with error compensation module (SAADI-EC), however *LEAD* has larger benefits in terms of area, power, and delay: 41.50%, 37.04%, and 20.14% respectively, over the SAADI-EC design, which makes *LEAD* as the hardware efficient and accurate accelerator division design. The accuracy of SAADI-EC is dependent on the allowable clock cycle operations thereby improving the precision of the results, with maximum accuracy reported for 7^{th} clock cycle for the 8-bit division. The *LEAD* design considering 2 clock cycles achieves accuracy results comparable to SAADI-EC configured for 7^{th} clock cycles. As reported in Table 3, *LEAD* design offers 10.2% less footprint, 65.48% less power and 57.93% improvement in delay when compared with SAADI-EC for t=15 iteration design. Around 51%, and 49% error improvements are offered for the *LEAD* divider over Logarithmic divider design.

The *LEAD* design for fixed-point and floating-point input data format for 16-bit division were also designed in similar methodology to yield output in fixed-point and floating-point respectively, and the results were compared. For the floating-point design, the input operands skipped the Normalization blocks, hence the design was expected to be hardware-efficient than other format designs. The *LEAD* design for integer formatted input operand was configured to generate output in floating-point format. Between Integer, fixed-point and floating-point division for 16-bit data-format, *LEAD* floating-point design was considered the best in terms of hardware

and error metrics as shown in the Table 3. The fixed-point output format generally losses the precision when compared to floating-point output, and hence the increase in error metrics. The *LEAD* divider design was inherently designed to generate floating-point output, hence no reconfiguration blocks was needed to reformat the output. Whereas the fixed-point format required the output to be reconfigured to fixed-point output, hence an added hardware cost is conceded by the *LEAD* design. The design is highly efficient for floating-type division.

For evaluating the divider design at application level, K-mean color quantization, that preserves appearance of the image while reducing the number of unique colours rendered to the image, was implemented. Based on the euclidean distance from cluster centroid, unique color is assigned to the clusters in the image, and the process is iterated several times by updating the mean of the clusters. The cluster averaging requires division operations at every iterations, where the hardware efficient division accelerator design is realized. Both 8-bit and 16-bit logical implementation of *LEAD*, Logarithmic, and SAADI-EC divider designs were applied individually for K-means color quantization operations in Matlab. The output of the color quantized images obtained from different divider designs were evaluated with three quality metrics - Peak-signal-to-noise-ratio (PSNR) in db, Structural similarity index (SSIM) in %, and Mean square error (MSE). The K-Mean color quantization algorithm with $K = 8$ was utilized to evaluate the divider designs. Tables 4, and 5 shows the quantization quality metrics for the original image as shown in the Figure 5 (a). For evaluating the divider performance, the initial center of the cluster was randomized for each of 10 runs, whereas the same initial cluster center was maintained for all the quantization implementations deduced from different dividers. Tables 4 and 5 shows that *LEAD* outperforms logarithmic divider in all three metrics. Additionally it performs better in terms of MSE and PSNR compared to SAADI-EC (t=7), and SAADI-EC (t=3) for 8-bit design, which is considered an accurate hardware divider. The SSIM index are comparable for both SAADI-EC (t=7), and *LEAD* divider design. However the performance of 16-bit *LEAD* design is comparable with that of SAADI-EC (t=3), and SAADI-EC (t=15). The quantization metrics for both 8-bit and 16-bit *LEAD* designs outperforms the logarithmic design. Between the *LEAD* data-format variants, the floating-point design is preferred over Integer and fixed-point format owing to slight improvements in MSE, PSNR, and SSM metrics. Figure 5 shows the output of color quantized image using k-mean clustering algorithm using different dividers separately. The k-mean method developed using *LEAD* divider design shows visually similar appearance as that of SAADI-EC (t=7) and exact divider.

4 CONCLUSIONS

The paper discusses about a novel divider accelerator design based on exponent series expansion, adopted to the logarithmic divider. The proposed divider offered better accuracy and performance than the existing divider designs. The proposed design yields faster results when compared with a divider which employs taylor series expansion with 3, 7 and 15 iterations. The accuracy of the proposed divider is equivalent to error compensated SAADI design, hence no additional error compensation module was required, leading to



Figure 5: K-mean color quantization algorithm when applied on (a) original image showing processed output image for individual dividers employed: (b) exact divider (c) 8-bit SAADI-EC(t=7) (d) 8-bit Logarithmic divider, and (e) 8-bit LEAD designs.

Table 4: Comparison of MSE, PSNR AND SSIM for color quantized output derived from different 8-bit divider designs.

Divider	MSE	PSNR (db)	SSIM
Accurate	129.043	27.0244	0.96048
Logarithmic	219.578	24.766	0.94306
LEAD	144.653	26.533	0.95906
SAADI-EC(t=3)	166.914	25.91772	0.95804
SAADI-EC(t=7)	153.649	26.26706	0.95972

Table 5: Comparison of MSE, PSNR AND SSIM for color quantized output derived from different 16-bit divider designs.

Divider	MSE	PSNR (db)	SSIM
Accurate	131.2008	26.95312	0.960442
Logarithmic	230.7066	24.54668	0.940768
SAADI-EC(t=15)	131.2397	26.95182	0.960446
SAADI-EC(t=3)	137.5566	26.75026	0.960433
LEAD	149.0771	26.40436	0.957458
LEAD-fixed	151.083	26.34574	0.957262
LEAD-floating	148.3129	26.42778	0.957758

large benefits in footprint, performance, and power savings. The proposed design was also minimally modified to generate 3 designs

catering to different input data-formats involving integer, fixed-point, and floating-point. The proposed hardware divider design is shown to be an hardware efficient, and accurate accelerator design compared to the SAADI and Logarithmic divider, both of which falters either in performance, or accuracy. The proposed divider showcased improved color quantization results when applied for K-means color clustering approach, as compared to other two dividers especially for 8-bit designs. Thus the proposed hardware accelerator divider is a step towards developing efficient system on chip design especially for image and video processing applications.

REFERENCES

- [1] Swagath Venkataramani, Srimat T. Chakradhar, Kaushik Roy, and Anand Raghunathan. Approximate computing and the quest for computing efficiency. In *2015 52nd ACM/EDAC/IEEE Design Automation Conference (DAC)*, pages 1–6, 2015.
- [2] Mark T. Bohr and Ian A. Young. Cmos scaling trends and beyond. *IEEE Micro*, 37(6):20–29, 2017.
- [3] Xuan Hu, Naimul Hassan, Wesley H. Brigner, Maverick Chauwin, and Joseph S. Friedman. Device modeling and circuit design for scalable beyond-cmos computing. In *2020 IFIP/IEEE 28th International Conference on Very Large Scale Integration (VLSI-SOC)*, pages 210–211, 2020.
- [4] Samiha Lubaba, K. M. Faisal, Moumita Sadia Islam, and Mehedi Hasan. Design of a two-bit magnitude comparator based on pass transistor, transmission gate and conventional static cmos logic. In *2020 11th International Conference on Computing, Communication and Networking Technologies (ICCCNT)*, pages 1–5, 2020.
- [5] Jie Han and Michael Orshansky. Approximate computing: An emerging paradigm for energy-efficient design. In *2013 18th IEEE European Test Symposium (ETS)*, pages 1–6, 2013.
- [6] Qiang Xu, Todd Mytkowicz, and Nam Sung Kim. Approximate computing: A survey. *IEEE Design Test*, 33(1):8–22, 2016.
- [7] Weiqiang Liu, Jing Li, Tao Xu, Chenghua Wang, Paolo Montuschi, and Fabrizio Lombardi. Combining restoring array and logarithmic dividers into an approximate hybrid design. In *2018 IEEE 25th Symposium on Computer Arithmetic (ARITH)*, pages 92–98, 2018.
- [8] Honglan Jiang, Cong Liu, Leibo Liu, Fabrizio Lombardi, and Jie Han. A review, classification, and comparative evaluation of approximate arithmetic circuits. *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, 13(4):1–34, 2017.
- [9] Yashaswi Mannepalli, Viraj Bharadwaj Korede, and Madhav Rao. *Novel Approximate Multiplier Designs for Edge Detection Application*, page 371–377. Association for Computing Machinery, New York, NY, USA, 2021.
- [10] K J N S Bhargav, Sairam Paliseti, and Madhav Rao. A newton raphson method based approximate divider design for color quantization application. In *2021 18th International SoC Design Conference (ISOC)*, pages 115–116, 2021.
- [11] Masoud Pashaeifar, Mehdi Kamal, Ali Afzali-Kusha, and Massoud Pedram. Approximate reverse carry propagate adder for energy-efficient dsp applications. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 26(11):2530–2541, 2018.
- [12] Udayan S. Patankar and Ants Koel. Review of basic classes of dividers based on division algorithm. *IEEE Access*, 9:23035–23069, 2021.
- [13] Jackson Melchert, Setareh Behroozi, Jingjie Li, and Younghyun Kim. Saadi-ec: A quality-configurable approximate divider for energy efficiency. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 27(11):2680–2692, 2019.
- [14] Setareh Behroozi, Jingjie Li, Jackson Melchert, and Younghyun Kim. Saadi: A scalable accuracy approximate divider for dynamic energy-quality scaling. In *Proceedings of the 24th Asia and South Pacific Design Automation Conference*, pages 481–486, 2019.
- [15] Lohita S. Niwal and S. P. Hajare. Design of radix 4 divider circuit using srt algorithm. In *2015 International Conference on Communications and Signal Processing (ICCS)*, pages 1107–1110, 2015.
- [16] Shaghayegh Vahdat, Mehdi Kamal, Ali Afzali-Kusha, Massoud Pedram, and Zainalabedin Navabi. Truncapp: A truncation-based approximate divider for energy efficient dsp applications. In *Design, Automation Test in Europe Conference Exhibition (DATE)*, 2017, pages 1635–1638, 2017.
- [17] Reza Zendegani, Mehdi Kamal, Arash Fayyazi, Ali Afzali-Kusha, Saeed Safari, and Massoud Pedram. Seerad: A high speed yet energy-efficient rounding-based approximate divider. In *2016 Design, Automation Test in Europe Conference Exhibition (DATE)*, pages 1481–1484, 2016.
- [18] Mohsen Imani, Ricardo Garcia, Andrew Huang, and Tajana Rosing. Cade: Configurable approximate divider for energy efficiency. In *2019 Design, Automation Test in Europe Conference Exhibition (DATE)*, pages 586–589, 2019.
- [19] John N. Mitchell. Computer multiplication and division using binary logarithms. *IRE Transactions on Electronic Computers*, EC-11(4):512–517, 1962.